Proportions: Pie, Donut, Treemap & Waffle

Workshop 3 — Module 5

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

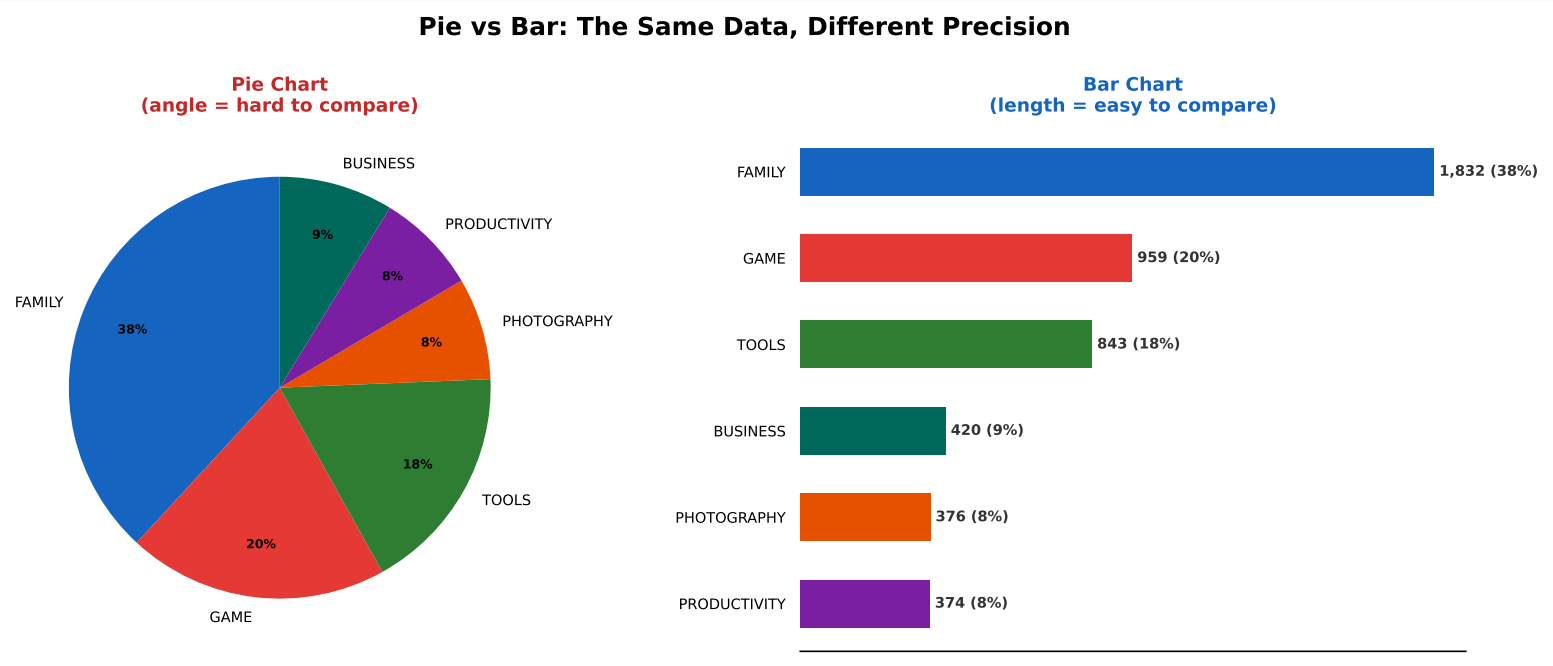
Module Roadmap

1. The pie chart debate: when pies work and when they fail
2. Donut charts: single-KPI highlights
3. Better alternatives: stacked bar and sorted bar with %
4. Treemaps: hierarchical part-to-whole
5. Waffle charts: intuitive unit-based proportions
6. Proportion chart decision tree

Learning Outcome
You will make an evidence-based decision about whether to use a pie chart (rarely) or a better alternative (usually), and produce treemaps and waffle charts for part-to-whole displays.

The Pie Chart Debate

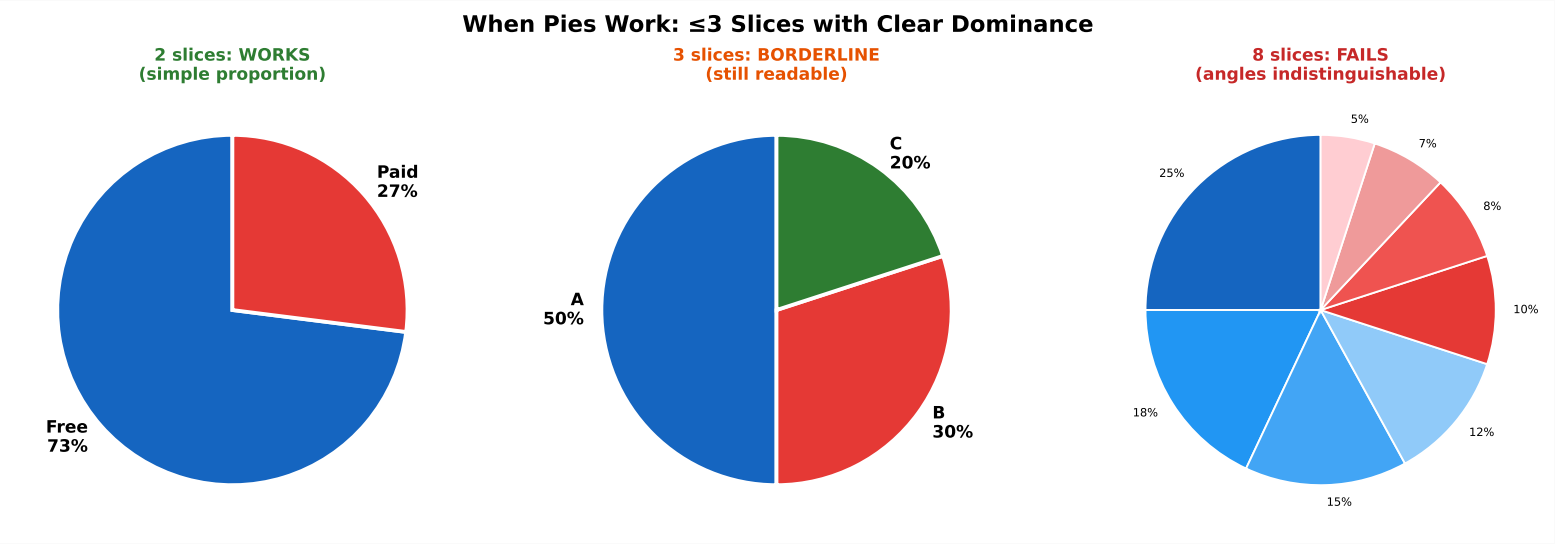
Pie vs Bar: Same Data, Different Precision



Pro-Tip

Angle is the **worst** perceptual channel (Cleveland & McGill, 1984). Humans are $\sim 2\times$ less accurate judging angles than lengths. A sorted horizontal bar with direct % labels always outperforms a pie chart for multi-category comparisons.

When Pies Work: $\leq$3 Slices

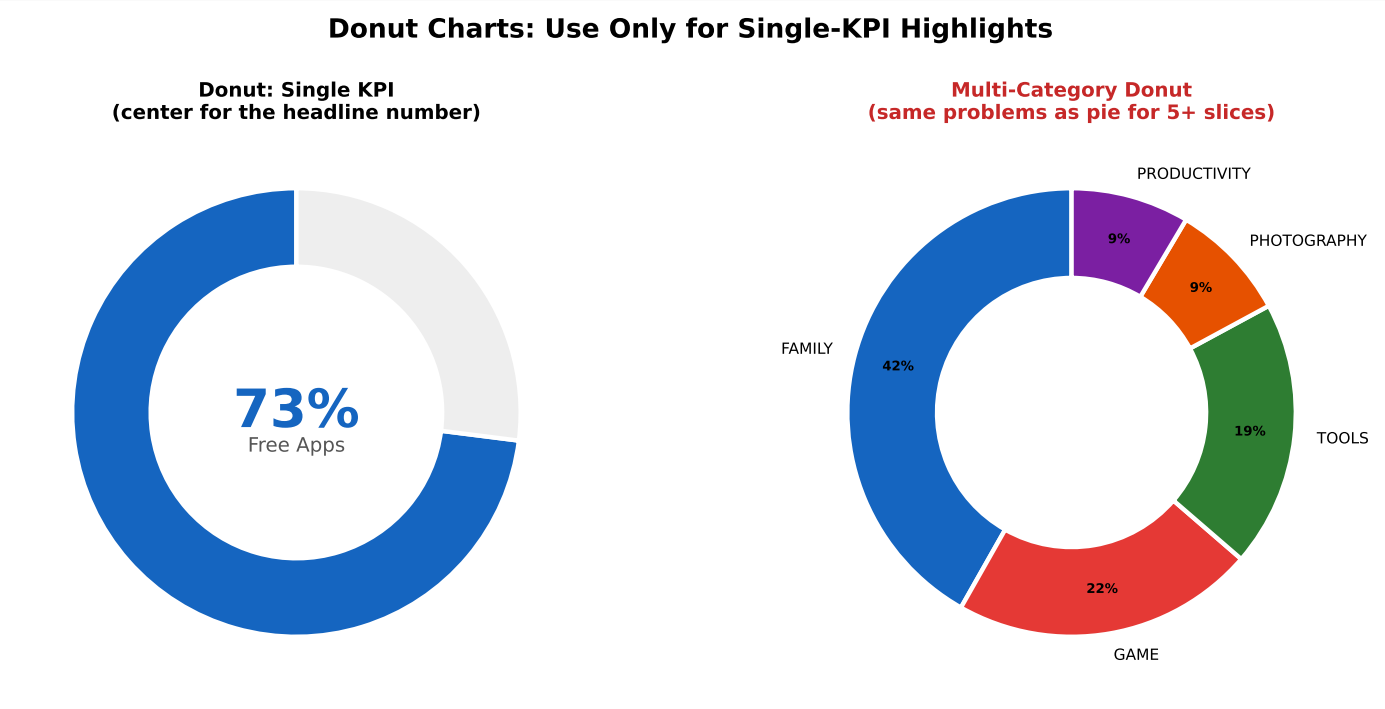


Data Insight

Pies work only when: (a) there are $\leq$3 slices, (b) one slice clearly dominates (e.g., 73% vs 27%), (c) the goal is to show "more than half" rather than precise comparison. For 4+ slices, the angle discrimination fails completely.

Donut Charts

Donut: Use Only for Single-KPI Highlights



Donut in R vs Python

R (ggplot2 + coord_polar)

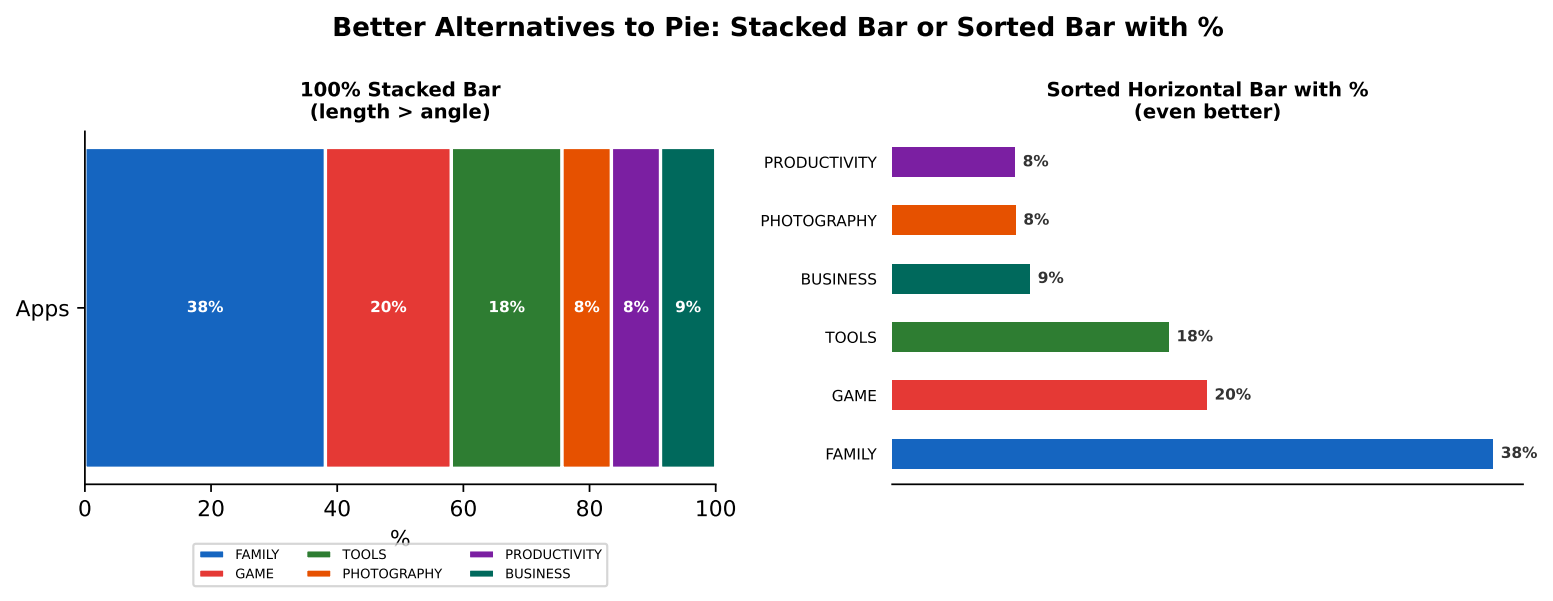
```r
# Single-KPI donut
df <- tibble(
  cat = c("Free", "Paid"),
  n = c(73, 27))

ggplot(df, aes(x = 2, y = n,
               fill = cat)) +
  geom_col(width = 1,
    color = "white", linewidth =
        2) +
  coord_polar(theta = "y") +
  xlim(0.5, 2.5) +   # hole
  scale_fill_manual(values =
    c(Free = "#1565C0",
      Paid = "#EEEEEE")) +
  annotate("text", x = 0.5, y =
      0,
    label = "73%", size = 10,
    fontface = "bold",
    color = "#1565C0") +
  theme_void() +
  theme(legend.position = "none")
```

Python

```python
fig, ax = plt.subplots()

wedges, _ = ax.pie(
    [73, 27],
    colors=["#1565C0", "#EEEEEE"
        ],
    startangle=90,
    wedgeprops={
        "width": 0.35,      # ring
        "edgecolor": "white",
        "linewidth": 2})

# Center text = the KPI
ax.text(0, 0, "73%",
    ha="center", va="center",
    fontsize=28, fontweight="bold
        ",
    color="#1565C0")
ax.text(0, -0.12, "Free Apps",
    ha="center", va="center",
    fontsize=10, color="#555")
```

Better Alternatives

Stacked Bar and Sorted Bar with %



100% Stacked Bar in Code

R

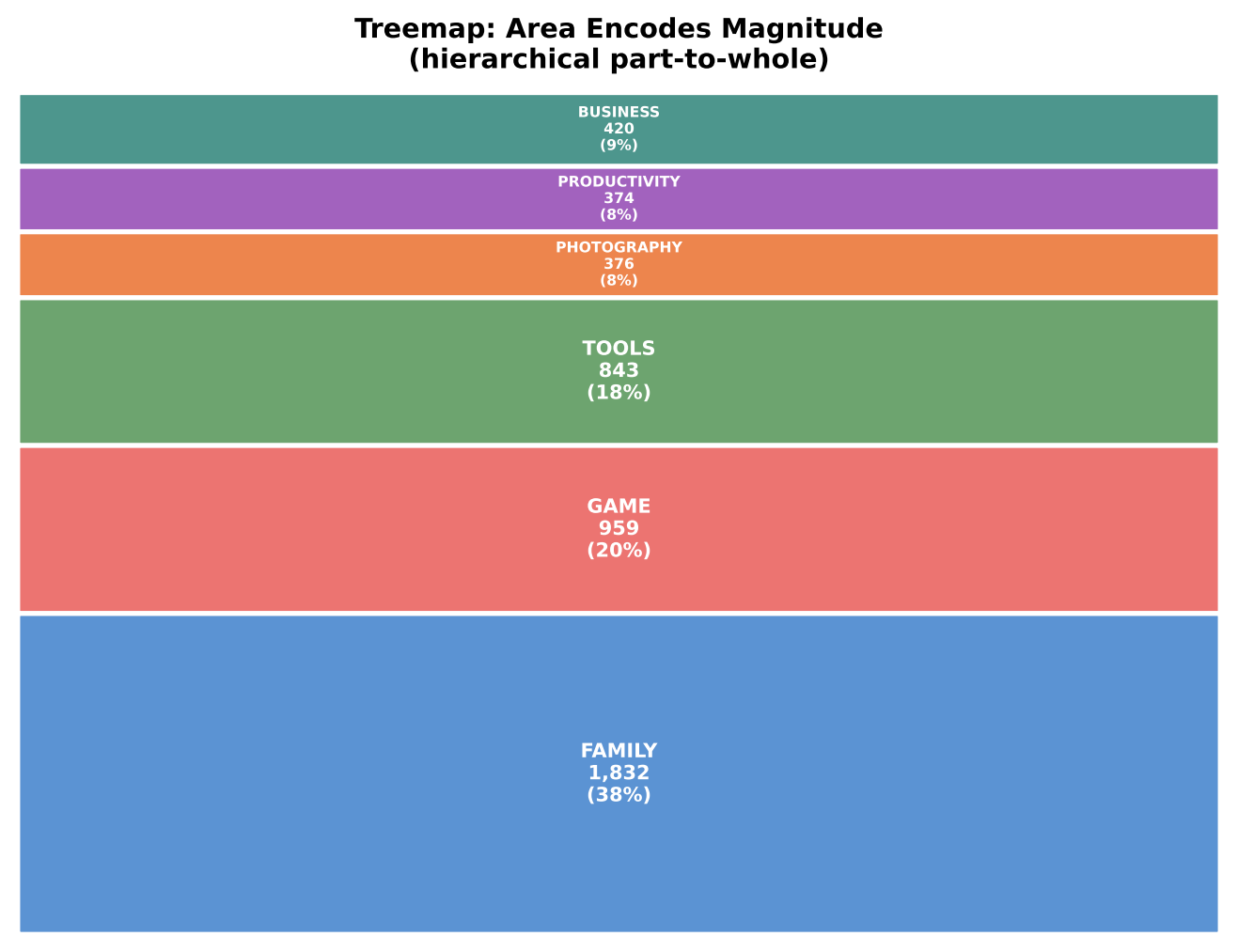
```r
# 100% stacked (fill)
ggplot(apps, aes(x = "Apps",
    fill = Category)) +
  geom_bar(position = "fill",
    width = 0.6) +
  geom_text(aes(
    label = paste0(
      round(..count../sum(..count
          ..)*
        100), "%")),
    stat = "count",
    position = position_fill(
      vjust = 0.5),
    size = 3, color = "white",
    fontface = "bold") +
  scale_y_continuous(
    labels = scales::percent) +
  coord_flip() +
  theme_minimal()
```

Python

```python
# Manual 100% stacked bar
left = 0
for cat, val, color in zip(
    cats, vals, colors):
    pct = val / sum(vals) * 100
    ax.barh(["Apps"], pct,
        left=left, color=color,
        height=0.5,
        edgecolor="white", lw
            =1.5,
        label=cat)
    if pct > 5:
        ax.text(left + pct/2, 0,
            f"{pct:.0f}%",
            ha="center", va="
                center",
            fontsize=7,
            fontweight="bold",
            color="white")
    left += pct
ax.set_xlim(0, 100)
ax.legend(fontsize=6, ncol=3)
```

Treemaps

Treemap: Area Encodes Magnitude



Data Insight

Treemaps encode part-to-whole via area, which is more accurate than angle (pie) but less accurate than length (bar). Their advantage is

Treemap in R vs Python

R (treemapify)

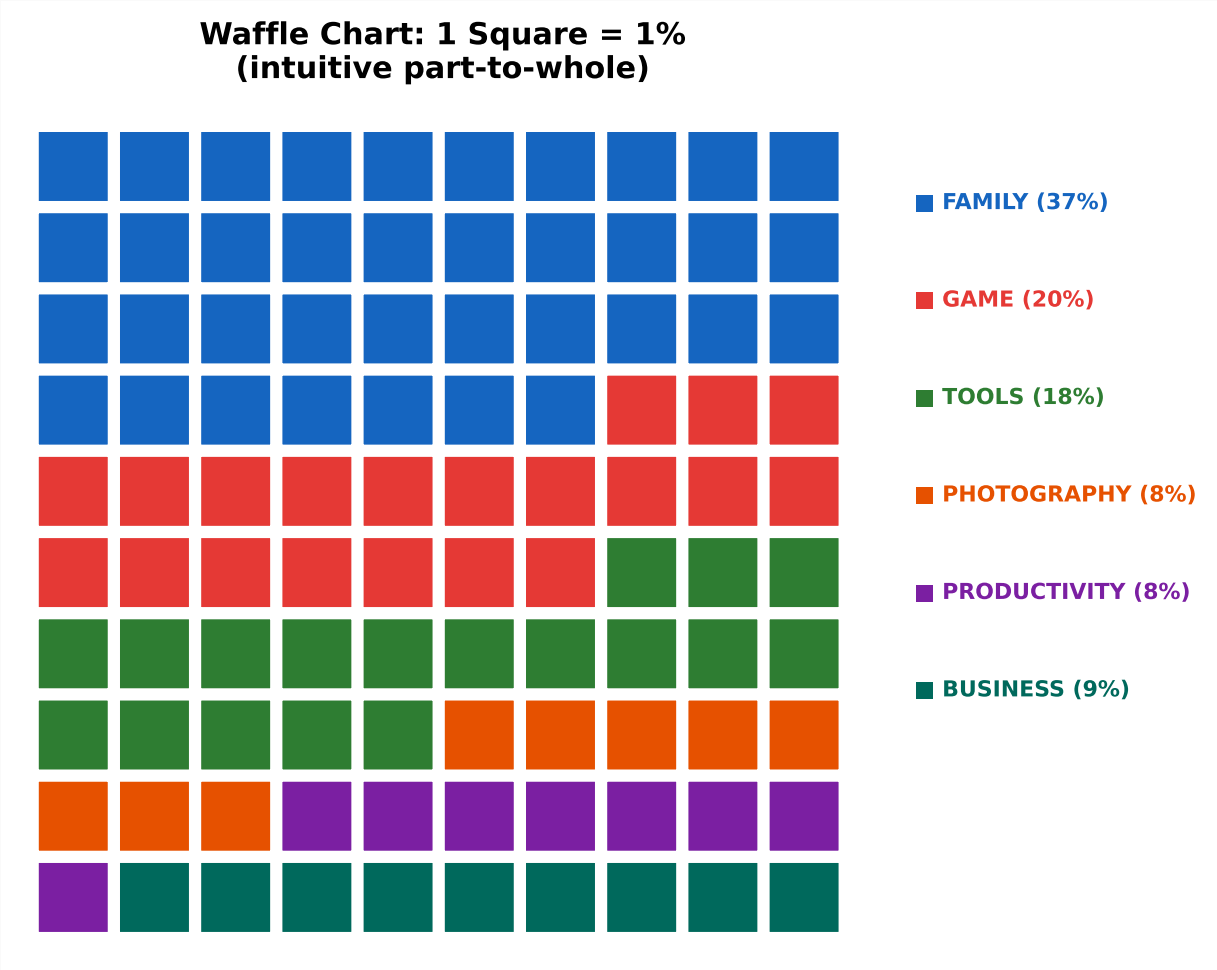
```r
library(treemapify)

ggplot(df, aes(
    area = n,
    fill = Category,
    label = paste(Category,
      scales::comma(n),
      sep = "\n"))) +
  geom_treemap(color = "white",
    size = 2) +
  geom_treemap_text(
    fontface = "bold",
    color = "white",
    place = "centre") +
  scale_fill_brewer(
    palette = "Set2") +
  theme(legend.position = "none")
      +
  labs(title = "App Categories")
```

Python (squarify)

```python
import squarify

fig, ax = plt.subplots(
    figsize=(8, 5))

squarify.plot(
    sizes=vals,
    label=[f"{c}\n{v:,}"
        for c, v in
        zip(cats, vals)],
    color=colors,
    alpha=0.7,
    edgecolor="white",
    linewidth=2,
    text_kwargs={
        "fontsize": 8,
        "fontweight": "bold",
        "color": "white"},
    ax=ax)

ax.axis("off")
ax.set_title("Treemap")
```

Waffle Charts

Waffle: 1 Square = 1%



Pro-Tip

Waffle charts use a 10×10 grid where each square represents 1%. They are more intuitive than pies because counting squares is easier than judging angles. In R: `waffle` package. In Python: `pywaffle` or manual

Waffle in R vs Python

R (waffle)

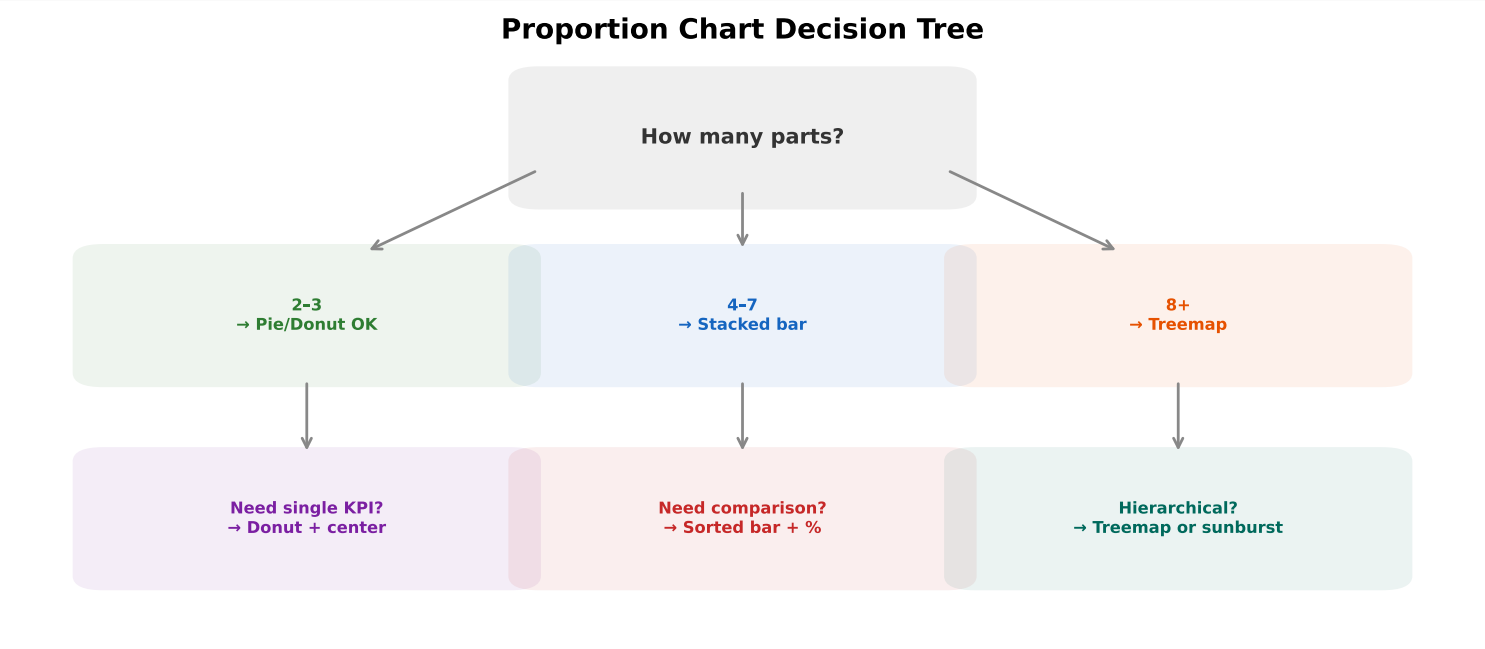
```r
# install.packages("waffle")
library(waffle)

waffle(
  c(FAMILY = 38, GAME = 20,
    TOOLS = 18, OTHER = 24),
  rows = 10,
  colors = c("#1565C0","#E53935",
    "#2E7D32","#BBBBBB"),
  title = "App Categories (%)",
  xlab = "1 square = 1%"
)
```

Python (manual)

```python
fig, ax = plt.subplots()
# Build 10x10 grid
grid = np.zeros((10,10), dtype=
    int)
idx = 0; cat_idx = 0
for i in range(10):
    for j in range(10):
        grid[9-i, j] = cat_idx
        idx += 1
        if (idx >= sum(pcts[:
            cat_idx+1])
            and cat_idx < len(
                pcts)-1):
            cat_idx += 1

# Draw squares
for i in range(10):
    for j in range(10):
        ax.add_patch(plt.
            Rectangle(
            (j, i), 0.9, 0.9,
            fc=colors[grid[i,j]],
            ec="white", lw=1.5))
ax.set_aspect("equal")
```

Decision Framework

Proportion Chart Decision Tree



Proportion Chart Comparison

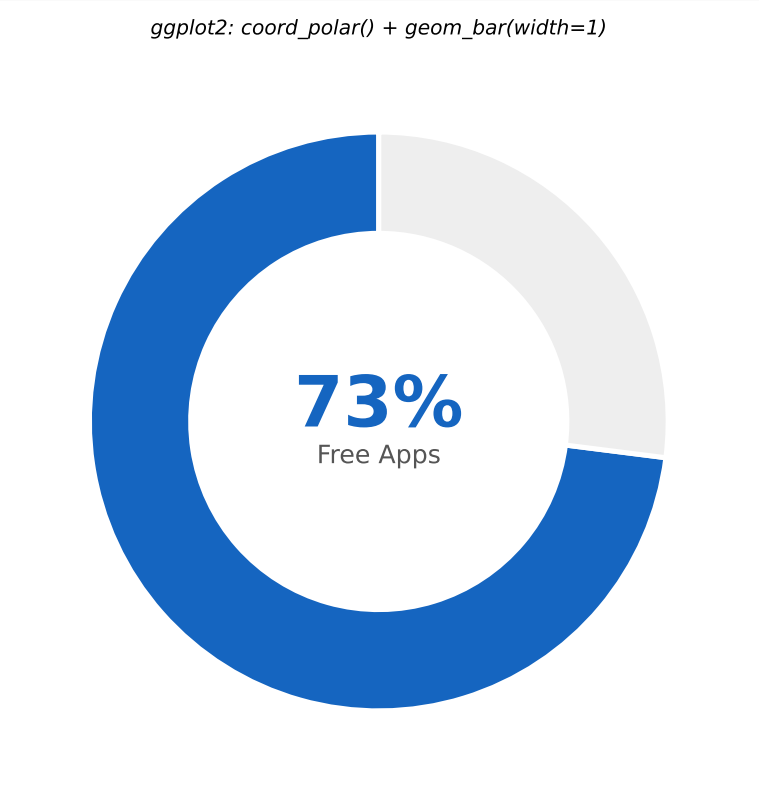
Proportion Chart Comparison: Strengths & Weaknesses

Chart	Best For	Weakness	R	Python
Pie	2–3 slices, clear dominance	Angle = worst perceptual channel	`geom_bar(coord_polar)`	`ax.pie()`
Donut	Single KPI highlight	Same as pie for multi-category	`(same + width)`	`ax.pie(width=0.35)`
Stacked bar	Part-to-whole + comparison	Only baseline is precise	`geom_bar(fill) + percent`	`ax.bar(bottom=)`
Treemap	Hierarchical part-to-whole	Area harder to compare than length	`treemapify`	`squarify`
Waffle	Intuitive % (1 unit = 1%)	Only for round % (max ~10 categories)	`waffle package`	`pywaffle / manual`

Complete Examples

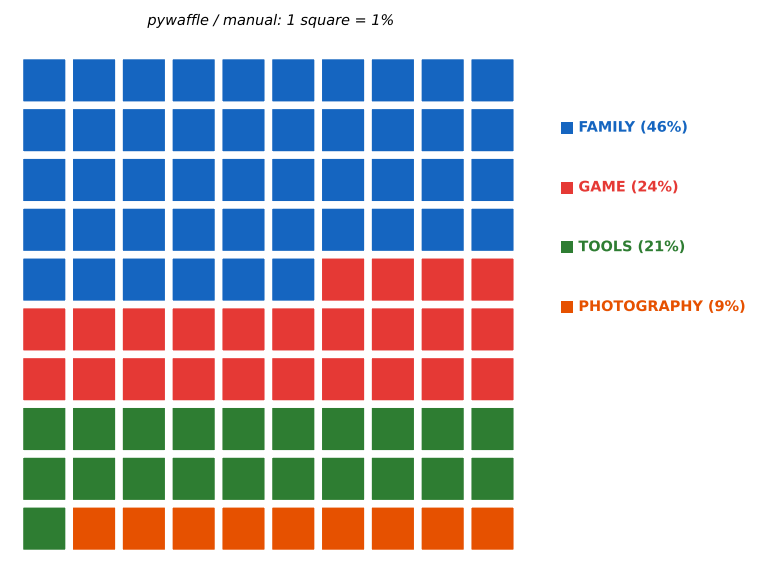
R: Single-KPI Donut

```r
# The only acceptable pie/donut:
# 2 slices + center KPI
df <- tibble(
  cat = c("Free", "Other"),
  n = c(73, 27))

ggplot(df, aes(x = 2, y = n,
               fill = cat)) +
  geom_col(width = 1,
    color = "white", lw = 2) +
  coord_polar(theta = "y") +
  xlim(0.5, 2.5) +
  scale_fill_manual(values =
    c(Free = "#1565C0",
      Other = "#EEEEEE")) +
  annotate("text", x = 0.5, y = 0,
    label = "73%", size = 12,
    fontface = "bold",
    color = "#1565C0") +
  annotate("text", x = 0.5, y = 5,
    label = "Free Apps",
    size = 4, color = "#555") +
  theme_void() +
  theme(legend.position = "none")
```



Python: Waffle Chart

```python
# Each square = 1%
# 10x10 grid = 100 squares

pcts = [38, 20, 18, 24]  # rounded
cats = ["FAMILY","GAME",
        "TOOLS","OTHER"]
colors = ["#1565C0","#E53935",
          "#2E7D32","#BBBBBB"]

fig, ax = plt.subplots(
    figsize=(6, 5))
# ... grid construction ...
# ... Rectangle loop ...

# Legend
for k, (cat, pct, c) in enumerate(
    zip(cats, pcts, colors)):
    ax.text(10.8, 8.5 - k*1.2,
        f"  {cat} ({pct}%)",
        fontsize=8, color=c,
        fontweight="bold")

ax.set_title(
    "1 square = 1%",
    fontweight="bold")
```



Synthesis

Key Takeaways

1. **Pie charts fail** for 4+ slices because angle is the worst perceptual channel. Use only for 2–3 slices with clear dominance.
2. **Donut charts** are pies with a hole — same problems, but acceptable for a single-KPI highlight with center text.
3. **100% stacked bar** or **sorted bar with %** almost always outperform pies for multi-category proportions.
4. **Treemaps** encode part-to-whole via area; best for hierarchical data with 10–50 nodes.
5. **Waffle charts** (1 square = 1%) are more intuitive than pies for general audiences.
6. Default to **sorted horizontal bar with %** — it is the safest proportion chart for any audience.

Essential Readings

- Cleveland, W. S. & McGill, R. (1984). "Graphical Perception." *JASA*, 79(387). (Angle $<$ length for proportion.)
- Spence, I. & Lewandowsky, S. (1991). "Displaying proportions and percentages." *Applied Cognitive Psychology*.
- Schwabish, J. (2021). *Better Data Visualizations*, Chapter 6 (Part-to-Whole).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 10 (Proportions).

Next Module: Rankings & Part-to-Whole (W03-M06)